COMPUTER NETWORKS · SEMESTER 1 · LECTURE 1

Introduction: what a network is and how the Internet is put together

Week 1 teaching pack: lecture notes, seminar, Linux laboratory and Windows laboratory (two hours each)

Associate Professor, PhD, Nicolaie Popescu-Bodorin, 2026,
+40722882471, bodorin(on)live.com

Part	Contents	Duration
A	Lecture 1 — Introduction: what a network is and how the Internet is put together	2 h
B	Seminar 1 — Reasoning about networks with numbers	2 h
C	Laboratory 1 (Linux) — Exploring the network from the command line	2 h
D	Laboratory 1 (Windows) — Exploring the network with PowerShell	2 h

Part A · Lecture 1

After this lecture you should be able to

- say what a computer network, a host, a link, a packet switch and a protocol are;
- describe the parts of the Internet: the edge, the access networks and the core;
- explain packet switching and circuit switching, and argue why the Internet uses the first;
- describe how thousands of independent networks are joined into one Internet;
- compute transmission and propagation delay for a simple path;
- name the five layers of the Internet protocol stack and say what each one does.

Plan of the two hours

Part	Topic	Time
Hour 1		
1	Why study networks; how this course works	10 min
2	What is a computer network? What is the Internet?	15 min
3	What is a protocol?	10 min
4	The network edge: hosts, access networks, physical media	15 min
	Break	10 min
Hour 2		
5	The network core: packet switching and circuit switching	20 min
6	A network of networks	10 min
7	A first look at delay	8 min

Part	Topic	Time
8	Layering in one picture	7 min
9	A short history; summary	5 min

1. Why study networks, and how this course works

Almost every program you will write in your career talks to another program on another machine. A mobile app calls a server; the server calls a database and three other services; each of those is replicated in several data centres. When such a system is slow, or fails only sometimes, or works in the lab and not for the customer, the cause is very often in the network between the parts. An engineer who understands that network can find the cause. One who treats it as a black box can only restart things and hope.

There is a second reason. The Internet is the largest engineered system people have built. It connects billions of devices, is owned by nobody, has no central control, and has kept working while growing by many orders of magnitude and while carrying applications its designers never imagined. The ideas that made this possible — layering, packet switching, best-effort delivery, putting intelligence at the edges — are worth knowing in their own right, and they reappear across computer science.

How the course is organised

- The course runs for two semesters of 14 weeks, with one two-hour lecture and one two-hour laboratory every week.
- **Semester 1** goes down the Internet protocol stack from the top: applications, then transport, then the network layer and routing. **Semester 2** covers the link and physical layers, wireless and mobile networks, security, and modern topics such as software-defined and data-centre networks.
- The laboratory is where you write network programs and take networks apart. You will use Linux, C and Python, and tools such as Wireshark. Laboratory attendance is compulsory.
- The main textbook is Kurose and Ross, *Computer Networking: A Top-Down Approach* (8th edition). Peterson and Davie, *Computer Networks: A Systems Approach*, is free online and is a good second view.
- Assessment each semester: written examination 50 %, laboratory 30 %, assignments 20 %. You need at least 5 in both the examination and the laboratory.

2. What is a computer network?

A **computer network** is a set of computing devices connected so that they can exchange data. That definition is broad on purpose. It covers two laptops joined by a cable, the Wi-Fi in this building, a mobile operator’s national network, and the Internet itself.

The **Internet** is a particular network: a worldwide network of networks, in which independently owned networks agree to use a common set of protocols — the TCP/IP family — so that any device on any of them can reach any device on any other. We can describe it in two ways.

2.1 The "nuts and bolts" view: what it is made of

- **Hosts**, also called **end systems**. These are the devices at the edge that run applications: laptops, phones, servers, and by now also televisions, cars, sensors and door locks. "Host" because they host application programs.
- **Communication links**. These carry bits between devices: copper cable, optical fibre, radio. A link's **transmission rate** (informally, its bandwidth) is measured in bits per second.
- **Packet switches**. A sending host cuts its data into chunks, adds a header to each, and sends the resulting **packets**. Packet switches receive packets on one link and forward them on another. The two main kinds are **routers**, used in the core of the network, and **link-layer switches**, used inside local networks.
- **Internet service providers (ISPs)**. Each ISP is itself a network of packet switches and links. Hosts reach the Internet through an ISP: a residential provider, a mobile operator, a university. In Romania, universities are connected through RoEduNet, the national research and education network.
- **Protocols**. Hosts and switches run protocols that control how information is sent and received. The two most important are **IP**, the Internet Protocol, and **TCP**, the Transmission Control Protocol.
- **Standards**. Protocols work only if everyone implements them the same way. Internet standards are developed by the Internet Engineering Task Force (IETF) and published as **RFCs** (Requests for Comments). Link-level standards such as Ethernet and Wi-Fi come from the IEEE 802 committees.

The sequence of links and switches a packet crosses from the sending host to the receiving host is called a **route** or **path**.

2.2 The service view: what it does for applications

From the point of view of a programmer, the Internet is an infrastructure that delivers data between application programs running on different hosts. Web browsing, messaging, video streaming and online games are **distributed applications**: they consist of several programs on several hosts that exchange data. Note that the applications run on the hosts, not on the routers. Routers move packets and know nothing about the Web or about video.

A program asks the Internet to deliver data through a programming interface called the **socket interface**. You can think of it as the postal rules: to send a letter you must put it in an envelope, write the address in a fixed format and put it in a post box. The post office then offers several services (ordinary, registered, express). The Internet likewise offers more than one delivery service, and choosing the right one for an application is a design decision we will return to. You will start programming with sockets in the laboratory in week 3.

2.3 Classifying networks

Two classifications appear everywhere in the literature and in job descriptions, so you should know the vocabulary.

By geographic scale	Typical extent	Example
PAN – personal area network	a few metres	phone and headphones over Bluetooth

By geographic scale	Typical extent	Example
LAN – local area network	a room, a building, a campus	the Ethernet and Wi-Fi of this faculty
MAN – metropolitan area network	a city	a cable or fibre operator's city network
WAN – wide area network	a country, a continent	an ISP backbone; RoEduNet

By **topology**, the shape in which nodes are connected: *bus* (all nodes on one shared cable, as in early Ethernet), *ring*, *star* (every node connected to a central device, as in today's switched Ethernet and in Wi-Fi), *tree*, and *mesh* (several alternative paths between nodes, as in the Internet core). The topology decides what happens when something fails. In a star, losing the centre disconnects everyone; in a mesh, traffic can go around a broken link.

By the **roles of the hosts**: in the **client–server** model a server that is always on waits for requests from clients (the Web works this way); in the **peer-to-peer** model hosts communicate directly and each acts as both client and server (BitTorrent works this way).

3. What is a protocol?

Consider asking a stranger for the time. You say "Excuse me." You wait. If they answer "Yes?", you ask "What time is it?" and they tell you. If they ignore you, or answer in a language you do not know, you do not ask the question; you give up or try someone else. There are specific messages, they come in a specific order, and each message received — or not received — triggers a specific action. That is a protocol.

Machines do the same, with one difference: nothing can be left to common sense. Everything must be specified.

> **Definition**
> A **protocol** defines the **format** and the **order** of the messages exchanged between two or more communicating entities, and the **actions** taken when a message is sent or received, or when some other event (such as a timeout) occurs.

Here is a real example. When you type a web address, your browser first sets up a TCP connection with the server (itself a three-message exchange), then sends a request in the HTTP protocol, and the server replies:

```
Browser → server:   GET /index.html HTTP/1.1
                    Host: www.example.org

Server → browser:   HTTP/1.1 200 OK
                    Content-Type: text/html
                    Content-Length: 1256

                    <!doctype html> ...
```

Every element here is fixed by the HTTP standard: the word GET, the spaces, the version string, the line endings, the meaning of the code `200`. A browser written in one country and a server written in another, by people who have never met, can talk to each other because both followed the same document.

Much of this course consists of studying protocols: **what** problem each one solves, **how** it solves it, and **why** it was designed that way and not another.

> **Question for the room**
> What should the browser do if no reply arrives? After how long? Should it send the request again? What if the request was "transfer 100 lei" and it was the reply that was lost, not the request?
> Keep this in mind. It is the central problem of the transport layer and we will spend three lectures on it.

4. The network edge

4.1 Hosts

Hosts are divided informally into **clients** and **servers**. Clients are the devices people use directly. Servers store and distribute content — web pages, video, e-mail — and today most of them live in large **data centres**, buildings that hold tens or hundreds of thousands of machines.

4.2 Access networks

The **access network** is the network that connects a host to the first router on its path, called the **edge router**. The main kinds are:

Access technology	How it works	Typical rates
DSL (digital subscriber line)	Uses the existing telephone pair. Data and voice travel on the same wire at different frequencies. The rate falls with distance from the exchange.	tens of Mbit/s down, less up
Cable (hybrid fibre–coax)	Uses the cable-television network. Fibre runs to a neighbourhood node and coaxial cable to the homes. The coax is a **shared medium**: neighbours share its capacity.	hundreds of Mbit/s to about 1 Gbit/s down
FTTH (fibre to the home)	Optical fibre all the way to the home, usually as a passive optical network in which one fibre from the provider is split among several homes. Widespread in Romanian cities.	1 Gbit/s and above
Ethernet	Twisted-pair copper cable from each host to a switch. The standard in offices, laboratories and data centres.	1–10 Gbit/s to the desk; up to 400 Gbit/s between switches
Wi-Fi (IEEE 802.11)	Hosts exchange radio signals with an access point within a few tens of metres; the access point is wired to the rest of the network.	tens of Mbit/s to over 1 Gbit/s
Cellular (4G, 5G)	Hosts exchange radio signals with a base station that may be kilometres away; the operator's network carries the data onward.	tens to hundreds of Mbit/s

The numbers are orders of magnitude, not promises; they change every few years. What does not change are two facts you should remember. Access links are often **asymmetric**, with more capacity towards the user than away. And many access media are **shared**, so the rate you get depends on what your neighbours are doing.

A typical home network combines several of these: a fibre terminal from the provider, a router that also acts as an Ethernet switch and a Wi-Fi access point, and a dozen devices behind it.

4.3 Physical media

A bit travelling from one host to another crosses a series of transmitter–receiver pairs. Between each pair it moves as an electromagnetic wave or a light pulse through a **physical medium**.

- **Guided media** confine the signal to a solid path. *Twisted-pair copper* is cheap and is what Ethernet cables are made of. *Coaxial cable* is used in cable-television networks. *Optical fibre* carries light pulses, supports enormous rates over long distances with very few errors, is immune to electrical interference, and is the medium of all long-distance links, including those under the oceans.
- **Unguided media** let the signal spread through the air or through space: Wi-Fi, cellular radio, and satellite links. Radio needs no cable, but the signal weakens with distance, is blocked and reflected by obstacles, and suffers interference from other transmitters.

We will study the physical layer in Semester 2. For now, one fact is enough: signals in fibre and copper travel at roughly two thirds of the speed of light in a vacuum, about 2×10^8 m/s.

5. The network core

The **core** is the mesh of routers and links that interconnects the access networks. There are two fundamentally different ways to move data through such a mesh.

5.1 Packet switching

The sending host breaks a long message into smaller **packets**. Each packet travels through links and packet switches to the destination, independently of the others. A packet is transmitted over each link at the **full rate of that link**.

Store and forward

Most packet switches use **store-and-forward transmission**: the switch must receive the *entire* packet before it can begin to transmit the first bit of it on the outgoing link.

Take a packet of L bits and a link of rate R bits per second. Pushing the packet onto the link takes L / R seconds. With one router between source and destination there are two links, so the packet arrives after $2L/R$ (ignoring, for now, the time the signal takes to travel). In general, across a path of N links of equal rate:

$$d = N \cdot L / R$$

> **Worked example 1**
> A 1500-byte packet is 12 000 bits. On a 100 Mbit/s link, L / R = 12 000 / 100 000 000 s = 0.12 ms. Across two such links (one router in between) the store-and-forward delay is 2 × 0.12 = 0.24 ms.

Queues and loss

Each switch has, for every outgoing link, an **output queue** (a buffer). If a packet arrives and the link is busy sending another one, the new packet waits in the queue. This waiting time is the **queueing delay**. It depends on how much traffic is arriving, and so it varies from one moment to the next.

Buffers are finite. A packet that arrives to a full queue has nowhere to go, and it — or one already in the queue — is dropped. This is **packet loss**. In the Internet, loss is not an exceptional fault. It is the normal consequence of congestion, and the protocols are designed to cope with it.

Forwarding and routing

How does a router know on which link to send a packet? Every packet carries the address of its destination in its header. Every router has a **forwarding table** that maps destination addresses (in fact, address prefixes) to outgoing links. The router looks up the destination and sends the packet on the link the table gives. This local action is **forwarding**.

The tables are filled in by **routing protocols**, by which routers exchange information and compute good paths through the whole network. This global process is **routing**. The analogy is a driver without a map who asks at every junction, “Which way to Cluj?”, and each time is pointed to the next road. Forwarding is following the sign at one junction. Routing is the work of deciding what the signs should say.

5.2 Circuit switching

The traditional telephone network worked differently. Before any data flows, the network sets up a **circuit**: it reserves capacity on every link along the path for the whole duration of the call. Once the circuit exists, the sender can transmit at a constant guaranteed rate, with no queueing and no loss.

A link is divided among circuits in one of two ways. With **frequency-division multiplexing (FDM)** each circuit gets its own frequency band all the time, as radio stations do. With **time-division multiplexing (TDM)** time is cut into frames and each frame into slots, and each circuit gets the same slot in every frame.

> **Worked example 2**
> A link of 2.048 Mbit/s uses TDM with 32 slots, so each circuit gets 2 048 000 / 32 = 64 kbit/s. To send a 640 000-bit file over one circuit takes 640 000 / 64 000 = 10 s. If setting up the circuit takes 0.5 s, the total is 10.5 s — regardless of whether the other 31 slots are in use or idle.

The last remark is the weakness. Reserved capacity that is not being used is **wasted**: nobody else may use it. That is acceptable for a telephone call, in which sound flows steadily. It is a poor fit for computers, whose traffic is **bursty**: a person reading a web page sends nothing for thirty seconds and then, on a click, wants a megabyte at once.

5.3 Packet switching versus circuit switching

With packet switching nothing is reserved. Capacity is given to whoever has a packet to send at that moment. This sharing on demand is called **statistical multiplexing**. An example shows how much it gains.

Worked example 3: how many users can share a link?

A link has a rate of 1 Gbit/s. Each user, when active, sends at 100 Mbit/s, but is active only 10 % of the time.

Circuit switching. 100 Mbit/s must be reserved for each user permanently. The link supports 1000 / 100 = **10 users**.

Packet switching. Admit 35 users. Trouble arises only when 11 or more are active at the same instant, because then they need more than 1 Gbit/s. Each user is active independently with probability 0.1, so the number of active users follows a binomial distribution, and P(11 or more active out of 35) = Σ (k = 11 … 35) C(35, k) × 0.1^k × 0.9^(35 − k) ≈ 0.0004. So with **35 users** — three and a half times as many — demand exceeds capacity only about 0.04 % of the time, and when it does, packets wait briefly in a queue and nothing worse happens.

	Packet switching	Circuit switching
Set-up before sending	none	required
Capacity	shared on demand	reserved per connection
Idle capacity	available to others	wasted
Delay	variable; queueing possible	constant after set-up
Under overload	delay grows, packets are lost	new connections are refused
Guarantees	none (“best effort”)	guaranteed rate
Best suited to	bursty data traffic	steady, constant-rate traffic

The Internet is packet-switched. It is simpler (no per-connection state in the routers), cheaper and far more efficient for bursty traffic. The price is that it gives **no guarantees**: packets may be delayed, lost, or arrive out of order. Repairing that, where an application needs it repaired, is the job of the hosts and not of the network. This division of labour is one of the Internet’s defining design choices, and we return to it next week under the name *end-to-end argument*.

6. A network of networks

Hosts connect to access ISPs. For any two hosts in the world to communicate, the access ISPs must themselves be interconnected. How?

Connecting every access ISP directly to every other is impossible: with hundreds of thousands of networks the number of links would grow with the square of that number. The structure that actually emerged was shaped as much by economics as by engineering, and it has several levels.

- **Transit.** A small ISP pays a larger one to carry its traffic to the rest of the Internet. The small ISP is the *customer*, the larger one the *provider*. Regional ISPs buy transit from national ones, and those from global ones.
- **Tier-1 ISPs.** At the top are roughly a dozen networks with global reach that pay nobody for transit. They exchange traffic with one another without payment and so, between them, can reach every destination.

- **Peering.** Two networks that exchange a lot of traffic can connect directly and carry each other's traffic, usually without payment. Both save the transit fee they would otherwise pay to a provider.
- **Internet exchange points (IXPs).** An IXP is a facility, in essence a large switch, at which many networks meet to peer with one another. Romania has several, for example InterLAN and RoNIX in Bucharest.
- **Multi-homing.** A network may buy transit from two or more providers, so that it stays connected if one of them fails.
- **Content provider networks.** Companies such as Google, Meta, Netflix and the content delivery networks operate private global networks linking their data centres. They peer directly with access ISPs and at IXPs, bypassing the upper tiers, to bring content close to users, lower their costs and control performance.

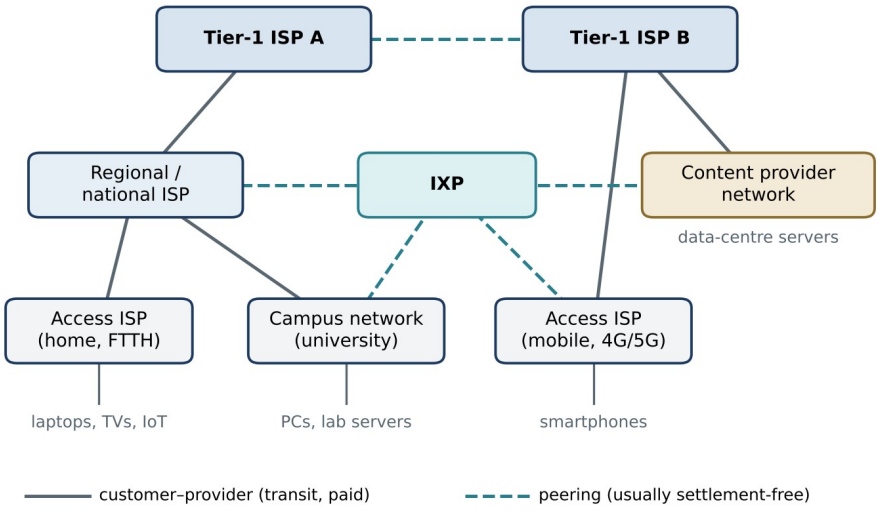


Figure 1. The Internet as a network of networks. Solid lines are paid customer–provider links; dashed lines are peering links.

The consequence to remember: a packet from your phone to a server typically crosses several independently operated networks. None of them controls the whole path, and none is obliged to give your packet any particular treatment. You will see this for yourself in the first laboratory with the `traceroute` tool.

7. A first look at delay

As a packet travels from one node to the next it is delayed for four different reasons.

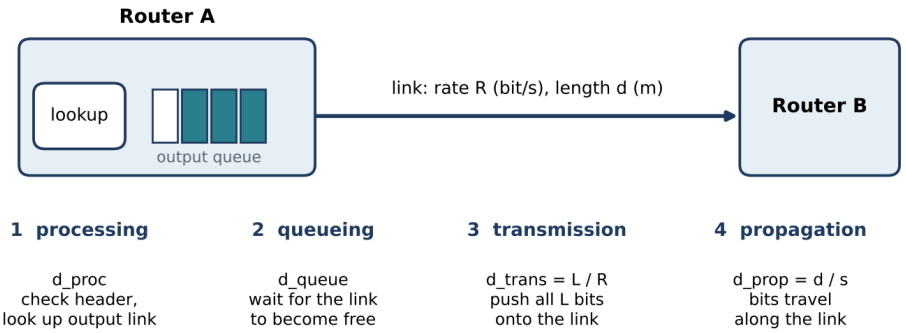


Figure 2. The four sources of delay at one hop.

1. **Processing delay.** The router examines the header, checks for bit errors and decides the outgoing link. On modern routers this takes microseconds.
2. **Queueing delay.** The packet waits for the link to be free. From zero to many milliseconds, depending on congestion.
3. **Transmission delay**, L / R. The time to push all L bits of the packet onto a link of rate R. It depends on the packet length and on the link rate, and **not** on the length of the link.
4. **Propagation delay**, d / s. The time one bit takes to travel a link of length d at signal speed s (about 2×10^8 m/s). It depends on distance, and **not** on the packet length or the link rate.

The total delay at one node is the sum: **d_nodal = d_proc + d_queue + d_trans + d_prop**.

Students confuse transmission and propagation delay more than anything else in this chapter. A convoy of ten cars on a motorway makes the distinction clear. Each car takes 12 seconds to pass through a toll booth, so the whole convoy needs 2 minutes to get through: that is transmission delay, and it depends on the number of cars and the speed of the booth. The cars then drive 100 km at 100 km/h, which takes one hour: that is propagation delay, and it depends on the distance and the speed of travel. A faster toll booth does not shorten the drive. A shorter road does not speed up the booth.

> **Worked example 4**
> A 1500-byte packet (12 000 bits) is sent from Cluj-Napoca to Bucharest over a fibre route of about 450 km, on a 100 Mbit/s link.
> Transmission delay: 12 000 / 100 000 000 s = 0.12 ms.
> Propagation delay: 450 000 m / 200 000 000 m/s = 2.25 ms.
> Here propagation dominates, by a factor of almost 20. On a slow link the opposite is true: at 1 Mbit/s the transmission delay of the same packet would be 12 ms.

Related to delay is **throughput**: the rate, in bits per second, at which data actually arrives at the receiver. Along a path, throughput is limited by the slowest link, the **bottleneck**. An 8-megabyte file is 64 million bits; if the slowest link on the path gives you 2 Mbit/s, the transfer takes at least 64 / 2 = 32 seconds, however fast the other links are. Next week we treat delay, loss and throughput in more depth.

8. Layering in one picture

The Internet has to solve many problems at once: turning bits into signals, sharing a radio channel, finding a path across the world, recovering lost packets, giving meaning to the data. Solving them in

one monolithic piece of software would be unmanageable. The solution is **layering**. The work is divided into layers, each of which offers a service to the layer above and uses the service of the layer below, without either needing to know how the other is implemented.

The benefit is that a layer can be changed without touching the others. Your browser works unchanged over Ethernet, Wi-Fi or 5G, and Wi-Fi did not have to be redesigned when video streaming was invented.

Layer	Its job	Examples	Unit of data
5 Application	Gives meaning to the exchange between application programs.	HTTP, DNS, SMTP	message
4 Transport	Delivers data between *processes* on two hosts; may add reliability and congestion control.	TCP, UDP, QUIC	segment
3 Network	Moves packets between *hosts* across many networks; addressing and routing.	IP, ICMP; OSPF, BGP	datagram
2 Link	Moves a packet across *one link* to the neighbouring node.	Ethernet, Wi-Fi	frame
1 Physical	Moves individual bits across the medium.	fibre, copper, radio signalling	bit

As data goes down the stack at the sender, each layer adds its own **header** in front of what it received from the layer above. This is **encapsulation**. At the receiver each layer reads and removes its header and passes the rest up.

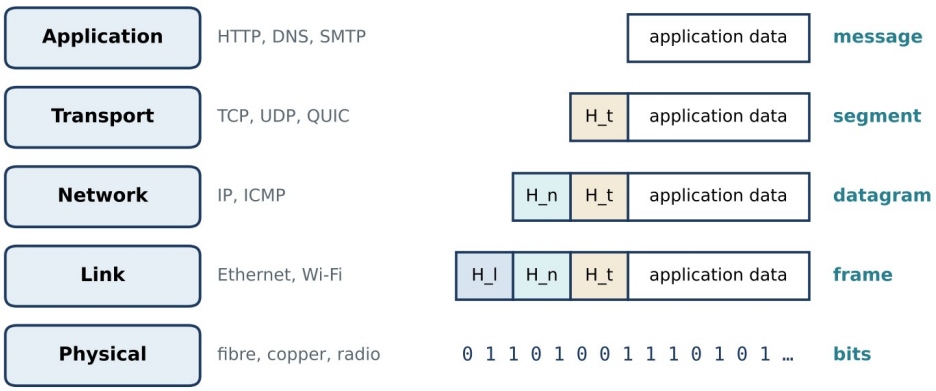


Figure 3. Encapsulation: each layer adds a header (H_t transport, H_n network, H_l link) to the data from the layer above.

Hosts implement all five layers. Routers implement only the bottom three, and link-layer switches the bottom two. This is the precise sense in which the Internet's core is simple and its intelligence sits at the edges. Next week we compare this five-layer model with the seven-layer OSI reference model.

9. A short history

Year	Event
1961–64	Leonard Kleinrock, Paul Baran and Donald Davies independently develop the idea of packet switching.

Year	Event
1969	The first four nodes of the ARPANET are connected: UCLA, the Stanford Research Institute, UC Santa Barbara and the University of Utah.
1971–72	Ray Tomlinson sends the first network e-mail; the ARPANET is demonstrated in public.
1974	Vinton Cerf and Robert Kahn publish the design of TCP and of an architecture for interconnecting different networks.
1983	On 1 January the ARPANET switches to TCP/IP. The Domain Name System is specified in the same year.
1986–88	NSFNET links American universities. After a series of congestion collapses, Van Jacobson adds congestion control to TCP.
1989–91	Tim Berners-Lee invents the World Wide Web at CERN: HTML, HTTP, the first server and the first browser.
1993	The Mosaic graphical browser appears. The `.ro` top-level domain is registered.
1995	NSFNET is retired; commercial providers carry the Internet backbone.
1998	IPv6 is specified (RFC 2460).
2007–10	Smartphones and mobile broadband put the Internet in everyone's pocket; large content networks reshape the topology.
2021–22	QUIC (RFC 9000) and HTTP/3 (RFC 9114) are standardised — a new transport, after forty years of TCP.

Two things stand out. The basic architecture of 1974–83 is still the one in use. And almost every application that matters today was invented later, by people who needed no permission from the network to deploy it.

Summary

- A network connects **hosts** by **links** and **packet switches**; the Internet is a network of networks that share the TCP/IP protocols.
- A **protocol** fixes the format and order of messages and the actions taken on sending, receiving and other events.
- Hosts reach the Internet through **access networks** (DSL, cable, fibre, Ethernet, Wi-Fi, cellular) over guided or unguided **physical media**.
- The core uses **packet switching** with store-and-forward routers. Packets queue, and may be lost, when links are busy. **Circuit switching** reserves capacity and gives guarantees, but wastes capacity on bursty traffic.
- **Forwarding** is a router's local action of moving a packet to the right output link; **routing** is the network-wide computation of the tables that make this possible.
- ISPs interconnect by **transit** and **peering**, often at **IXPs**; content providers run their own global networks.
- Delay has four components: processing, queueing, **transmission (L/R)** and **propagation (d/s)**. Throughput is limited by the bottleneck link.
- The protocol stack has five **layers**: application, transport, network, link, physical. Each adds its header: **encapsulation**.

Key terms

host / end system · link · transmission rate · packet · router · link-layer switch · ISP · protocol · RFC · socket · client–server · peer-to-peer · LAN / WAN · topology · access network · edge router · guided / unguided media · packet switching · store-and-forward · queueing delay · packet loss · forwarding table · routing protocol · circuit switching · FDM / TDM · statistical multiplexing · transit · peering · IXP · multi-homing · transmission delay · propagation delay · throughput · bottleneck · layer · encapsulation

Review questions

1. What is the difference between a host and a packet switch? Is a home Wi-Fi router a host?
2. State the three things a protocol defines. Identify each of them in the HTTP example of section 3.
3. Why are residential access links often asymmetric?
4. A router has received half of a packet. Why can it not yet start to forward it?
5. Give one application for which circuit switching would be a better fit than packet switching, and say why.
6. Explain the difference between forwarding and routing in your own words.
7. Why would two ISPs agree to carry each other's traffic for free?
8. Which delay component changes if you replace a 100 Mbit/s link with a 1 Gbit/s link of the same length? Which stays the same?
9. Which layers does a router implement? Why does it not need the others?

Exercises

1. Five packets of 1500 bytes each are sent one after another along a path of three links, each of 10 Mbit/s, through two store-and-forward routers. Ignoring propagation, processing and queueing, when does the last bit of the last packet arrive?
2. A geostationary satellite is about 36 000 km above the Earth. What is the propagation delay from a ground station up to the satellite and down to another ground station? (Radio travels at 3×10^8 m/s.) What does this mean for a telephone conversation?
3. A link of 1 Gbit/s is shared by users who each need 100 Mbit/s when active and are active 10 % of the time. With packet switching and 35 users, what is the probability that *all 35* are active at once? Compare it with the probability found in worked example 3 and comment.
4. You must move 40 terabytes of data from Iași to Bucharest. You have a dedicated 100 Mbit/s link. A courier can deliver disks in 8 hours. Which is faster?
5. How long is a packet, in metres? Consider a 12 000-bit packet on a 100 Mbit/s fibre link. (Hint: how far does the first bit travel before the last bit leaves the sender?)

Answers

1. $L / R = 12\,000 / 10\,000\,000$ s = 1.2 ms. The first packet arrives after $3 \times 1.2 = 3.6$ ms; each later packet arrives 1.2 ms after the one before. The fifth arrives at $(3 + 4) \times 1.2 =$ **8.4 ms**. In general, P packets over N links take $(N + P - 1) \cdot L / R$.

2. Up: 36 000 000 / 300 000 000 = 0.12 s. Up and down: **0.24 s**. A question and its answer are separated by about half a second of pure propagation delay, which people notice and which makes conversation awkward. Low-Earth-orbit satellites, at a few hundred kilometres, avoid this.
3. $0.1^{35} = 10^{-35}$, which is never, for practical purposes. The risk in statistical multiplexing does not come from everyone being active together. It comes from slightly more users than the link can carry being active, which is why the figure of 0.0004 is the relevant one.
4. 40 TB = 40×10^{12} bytes = 3.2×10^{14} bits. At 10^8 bit/s this takes 3.2×10^6 s, about **37 days**. The courier wins easily. Throughput and delay are different measures: the courier has enormous throughput and terrible delay.
5. The packet takes 0.12 ms to transmit. In that time the first bit travels 0.00012 s × 2×10^8 m/s = **24 km**. The packet is stretched along 24 km of fibre.

Reading and next steps

Required

- Kurose and Ross, *Computer Networking: A Top-Down Approach*, 8th edition, Chapter 1: sections on what the Internet is, the network edge, the network core, and the history of networking.

Recommended

- Peterson and Davie, *Computer Networks: A Systems Approach*, Chapter 1 (free at book.systemsapproach.org).
- Tanenbaum, Feamster and Wetherall, *Computer Networks*, 6th edition, Chapter 1.
- D. D. Clark, “The Design Philosophy of the DARPA Internet Protocols”, *Proc. ACM SIGCOMM*, 1988. Ten pages on why the Internet is built as it is, by one of its architects.

Laboratory 1 (this week)

Setting up the Linux environment. You will find your machine’s addresses and default gateway with `ip`, measure round-trip times with `ping`, and use `traceroute` to list the routers between the laboratory and several servers in Romania and abroad. Bring the answers to these two questions: how many networks does your packet cross, and where does most of the delay come from?

Next lecture

Layered architectures in detail: the OSI and TCP/IP reference models, encapsulation step by step, and a quantitative treatment of delay, loss and throughput.

PART B · WEEK 1

Seminar 1: Reasoning about networks with numbers

Two-hour problem session (2 × 50 minutes) accompanying Lecture 1

The seminar turns the ideas of the lecture into calculations and design decisions. Students work in pairs; after each problem one pair presents its solution at the board. No computers are needed, only a calculator.

By the end of the seminar you should be able to

- convert correctly between bytes and bits and between the usual rate units;
- compute transmission, propagation and end-to-end delay on a multi-hop path;
- compare message switching, packet switching and circuit switching quantitatively;
- explain with a calculation why statistical multiplexing works, and when it fails;
- specify a small protocol completely: messages, order, actions, errors.

Plan

Step	Activity	Time
A	Warm-up: units and orders of magnitude	10 min
B	Problems 1–2: delay on one link and on a path	25 min
C	Problem 3: why packets? Segmentation and its price	15 min
	Break	10 min
D	Problems 4–5: sharing a link; queues	25 min
E	Problem 6: design a protocol (group work)	20 min
F	Problems 7–8: encapsulation overhead; reading a traceroute	15 min
	Wrap-up and homework	(in F)

A. Warm-up: units

Agree on the conventions used all year, because most wrong answers in this chapter are unit errors.

- 1 byte = 8 bits. File sizes are given in **bytes** (B); link rates in **bits per second** (bit/s). Always convert to bits before dividing.
- For rates the prefixes are decimal: 1 kbit/s = 10^3 bit/s, 1 Mbit/s = 10^6 bit/s, 1 Gbit/s = 10^9 bit/s.
- In this course file sizes are also decimal (1 MB = 10^6 bytes) unless stated. Operating systems often report binary units (1 MiB = 2^{20} bytes); the difference is about 5 % at the megabyte level.
- Signal speed in copper and fibre: $s \approx 2 \times 10^8$ m/s = 200 km per millisecond. In air and vacuum: 3×10^8 m/s.

Quick questions (answer aloud). (a) How many bits is a 1500-byte packet? (b) How long does a 5 MB file take at 20 Mbit/s? (c) How far does a signal travel in fibre in 1 ms? (d) A phone shows “download: 4 MB/s”. What is that in Mbit/s?

Solution

(a) 12 000 bits. (b) $40 \times 10^6 / 20 \times 10^6 = 2$ s. (c) 200 km. (d) 32 Mbit/s.

B. Delay

Problem 1 — one link

Hosts A and B are joined by a single link of rate R = 10 Mbit/s and length 2000 km. The signal speed is 2×10^8 m/s. A sends one packet of 1500 bytes.

1. Compute the transmission delay and the propagation delay.
2. When does the last bit arrive at B?
3. At the instant A finishes transmitting, where is the first bit?
4. For what link length would the two delays be equal?
5. How long, in metres, is one bit on this link?

Solution

1. $d_{trans} = 12\,000 / 10^7$ s = 1.2 ms. $d_{prop} = 2 \times 10^6$ m / 2×10^8 m/s = 10 ms.
2. The last bit leaves at 1.2 ms and travels for 10 ms: it arrives at 11.2 ms.
3. It has travelled for 1.2 ms: 1.2 ms × 200 km/ms = 240 km from A. The whole packet occupies 240 km of the link.
4. $d / s = L / R$ gives $d = s \cdot L / R$ = 240 km.
5. One bit lasts $1 / R$ = 0.1 µs and so occupies s / R = 20 m.

Problem 2 — a path of three links

A packet of 12 000 bits goes from host A through routers R1 and R2 to host B. Each router needs 20 µs of processing; the queues are empty.

Link	Rate	Length
A – R1 (campus LAN)	100 Mbit/s	1 km
R1 – R2 (regional link)	10 Mbit/s	500 km
R2 – B (data-centre link)	1 Gbit/s	50 km

1. Compute the end-to-end delay of the packet.
2. Which component dominates, and on which link?
3. What is the maximum throughput of a long file transfer from A to B? How long does a 30 MB file take?
4. The operator upgrades the first link to 1 Gbit/s. By how much does the file transfer speed up?

Solution

1. Transmission: 0.12 + 1.2 + 0.012 = 1.332 ms. Propagation: 0.005 + 2.5 + 0.25 = 2.755 ms. Processing: 2 × 0.02 = 0.04 ms. Total ≈ 4.13 ms.
2. Propagation on the 500 km link (2.5 ms), followed by transmission on the same link (1.2 ms). The slow, long link dominates both.
3. The bottleneck is 10 Mbit/s. 30 MB = 240×10^6 bits, so at least 24 s.
4. Not at all: the bottleneck is unchanged. Upgrading a link that is not the bottleneck does not raise throughput.

C. Why packets?

Problem 3 — segmentation

A message of 1.2×10^6 bits must cross three links of 2 Mbit/s each, through two store-and-forward switches. Ignore propagation and processing.

1. **Message switching.** The message is sent whole. When does it arrive?
2. **Packet switching.** The message is cut into 100 packets of 12 000 bits. When does the last packet arrive?
3. Each packet needs a 40-byte header. Recompute part 2 and give the overhead as a percentage.
4. Give two further advantages and one disadvantage of cutting messages into packets.

> **Solution**
> 1. Each hop takes $1.2 \times 10^6 / 2 \times 10^6 = 0.6$ s. Three hops: 1.8 s.
> 2. $L / R = 6$ ms. Using $(N + P - 1) \cdot L / R$ with $N = 3$ links and $P = 100$ packets: 102×6 ms $= 0.612$ s — almost three times faster, because the three links work in parallel on different packets.
> 3. Packets are now 12 320 bits, $L / R = 6.16$ ms, total $102 \times 6.16 = 0.628$ s. Overhead: 320 / 12 320 ≈ 2.6 %.
> 4. Advantages: a bit error forces retransmission of one small packet and not of the whole message; a long message does not block a switch for everyone else. Disadvantage: header overhead, and packets can arrive out of order and must be reassembled.

D. Sharing a link

Problem 4 — circuits or packets?

A link of 100 Mbit/s serves users who each transmit at 10 Mbit/s when active and are active 20 % of the time, independently of one another.

1. How many users can circuit switching support?
2. With packet switching and 30 users, what is the average number of active users and the average load on the link?
3. Write the expression for the probability that the 30 users together demand more than the link can carry. Its value is about 0.026. Interpret it.
4. Would you admit 30 users? What would you want to know before deciding?

> **Solution**
> 1. 100 / 10 = 10 users.
> 2. $30 \times 0.2 = 6$ active users on average, a load of 60 Mbit/s: 60 % of capacity.
> 3. $P(\text{more than 10 active}) = \Sigma\ (k = 11 \ldots 30)\ C(30, k) \times 0.2\text{^}k \times 0.8\text{^}(30 - k) \approx 0.026$. About 2.6 % of the time demand exceeds capacity and packets queue.
> 4. It depends on the application. For web browsing, short queueing episodes 2.6 % of the time are acceptable, and three times as many customers are served. For a voice call or a remote-surgery link they may not be. Compare with the lecture example (35 users active 10 % of the time gave 0.0004): how bursty the users are matters more than how many they are.

Problem 5 — a queue forms

A router's outgoing link has a rate of 1 Mbit/s. All packets are 12 000 bits long.

1. Five packets arrive at the same instant to an empty queue. What is the queueing delay of each, and the average?
2. Packets arrive one every 12 ms. What is the queueing delay?
3. Packets arrive one every 10 ms. What happens? If the buffer holds 20 packets, roughly when is the first packet dropped?
4. Define the ratio (arrival rate in bit/s) / (link rate) for cases 2 and 3. What does the value 1 mean?

Solution

1. $L / R = 12$ ms. The packets wait 0, 12, 24, 36 and 48 ms; the average is 24 ms. In general the n-th packet waits $(n - 1) \cdot L / R$.

2. Zero. Each packet finishes exactly as the next arrives.

3. 100 packets/s arrive but only 83.3 packets/s can leave, so the queue grows by about 16.7 packets every second. A 20-packet buffer fills in about 1.2 s; after that roughly one packet in six is dropped.

4. Case 2: 1.0. Case 3: 1.2. This ratio is the **traffic intensity**. Above 1 the queue grows without limit, whatever the buffer size; a bigger buffer only delays the loss and adds delay. We study it next week.

E. Design a protocol

Problem 6 — group work

In groups of three or four, design an application protocol for a **time service**: a client asks a server what time it is. You have 12 minutes; then two groups present and the others try to break their design. Your specification must answer every question below, because a program cannot guess.

- What exactly does the client send? Text or binary? How does the server know where the message ends?
- What exactly does the server reply? In what format and time zone is the time?
- What does the server do with a message it does not understand?
- What does the client do if no reply comes? After how long? How many times does it try?
- Is it harmful if the server receives the same request twice?

One possible answer

Messages are lines of ASCII text ending in CR LF. Request: `TIME?`. Reply: `TIME 2026-10-06T09:30:00Z` (ISO 8601, always UTC). Error reply: `ERR 400 unknown request`.

Order. The client speaks first; the server sends exactly one reply per request.

Actions. The server answers any line other than `TIME?` with `ERR 400` and stays ready. The client waits 2 s for a reply, then repeats the request, at most three times, then reports failure to the user.

Duplicates are harmless: asking the time twice changes nothing on the server. Such a request is called **idempotent**. A request like “transfer 100 lei” is not, and repeating it blindly is dangerous; it needs a request identifier so the server can recognise a repetition. We return to this in the transport-layer lectures.

Points to draw out in the discussion: every group forgot something (usually the end-of-message rule or the timeout); real RFCs are long for exactly this reason; and the three parts of the lecture's definition — format, order, actions — give a checklist for any protocol.

F. Layers and paths

Problem 7 — the cost of headers

An application sends a 100-byte message over TCP (20-byte header), IPv4 (20-byte header) and Ethernet (14-byte header and 4-byte trailer).

1. How long is the frame, and what fraction of it is overhead?
2. Repeat for a 1460-byte message.
3. Which headers does a link-layer switch read? A router? The destination host?

> **Solution**
>
> 1. 100 + 20 + 20 + 18 = 158 bytes; overhead 58 / 158 ≈ 37 %.
> 2. 1518 bytes; overhead 58 / 1518 ≈ 3.8 %. Small messages are expensive, which is one reason applications batch data.
> 3. The switch reads only the Ethernet header. The router reads the Ethernet header and the IP header. The destination host reads all of them, and its application reads the message.

Problem 8 — reading a traceroute

The output below is invented for the exercise. Each line is one router on the path, with three round-trip time measurements.

```
1  gw.campus.example.edu (10.20.0.1)          0.4 ms   0.3 ms   0.4 ms
2  core1.campus.example.edu (10.0.0.1)        0.7 ms   0.6 ms   0.7 ms
3  cta-1.nren.example.net (192.0.2.17)        1.9 ms   1.8 ms   2.1 ms
4  buc-2.nren.example.net (192.0.2.41)        5.2 ms   5.1 ms   5.4 ms
5  * * *
6  vie-1.transit.example.net (198.51.100.9)  22.8 ms  23.1 ms  22.7 ms
7  fra-3.transit.example.net (198.51.100.33) 34.6 ms  34.2 ms  61.0 ms
8  nyc-1.transit.example.net (198.51.100.77) 118.3 ms 117.9 ms 118.6 ms
9  edge.cdn.example.com (203.0.113.5)        117.5 ms 118.1 ms 117.8 ms
```

1. How many networks does the packet appear to cross? How can you tell?
2. Between which two hops does the path cross the Atlantic? Estimate the length of that link.
3. What could the line `*  *  *` mean? Is the path broken there?
4. At hop 7 one measurement is 61 ms. Give two explanations.
5. Hop 9 shows a slightly smaller delay than hop 8. Is that possible?

Solution

1. Four: the campus (hops 1–2), the research network (3–4), a transit provider (6–8) and the content provider (9). The change of domain name and of address block shows the boundaries.
2. Between hops 7 and 8: the round-trip time jumps by about 84 ms, so one way is about 42 ms, which is 42 × 200 = 8400 km of fibre at most. That is consistent with Frankfurt to New York by cable.
3. The router at hop 5 did not answer the probes; many routers are configured not to, or limit how often they do. The path is not broken, because later hops answer.
4. A momentary queue on the path, or the router itself being slow to generate the reply: answering probes is low-priority work for a router. One slow sample among three is noise.
5. Yes. Each line is measured with different packets at a different moment, and the time a router takes to generate a reply varies. Only large, consistent differences mean something.

Homework

- Solve exercises 1–5 at the end of the lecture notes without looking at the answers.
- Repeat Problem 4 for users who are active 50 % of the time, with 15 users. Is statistical multiplexing still worthwhile?
- Find RFC 867 (“Daytime Protocol”), which is one page long, and compare it with your design from Problem 6. What did its author leave unspecified?

PART C · WEEK 1

Laboratory 1 (Linux): Exploring the network from the command line

Two-hour laboratory sheet (2 × 50 minutes) accompanying Lecture 1

In this laboratory you examine a real host and the real Internet with the standard Linux tools. Everything you saw in the lecture — interfaces, links, routers, protocols, delay, the network of networks — will appear on your screen.

By the end of the laboratory you should be able to

- find a host's interfaces, addresses, default gateway and DNS servers;
- test reachability and measure round-trip time with `ping`;
- discover the routers on a path with `traceroute` and `mtr`, and recognise the networks crossed;
- translate names to addresses with `dig`;
- speak a protocol by hand over a TCP connection with `nc`, and watch it with `curl -v`;
- measure throughput and relate it to the bottleneck link.

Before you start

- **Environment.** The course virtual machine (Ubuntu or Debian), a native Linux installation, or WSL 2 on Windows. All commands are typed in a terminal.
- **Packages.** If a command is missing, install the set once:

```
sudo apt update
sudo apt install -y iproute2 iputils-ping traceroute mtr-tiny dnsutils curl netcat-openbsd tcpdump
```

- **Getting help.** `man ping`, `ping --help`, and `ip addr help`. Reading the manual page is part of the work.
- **Rules of conduct.** Probe only the laboratory network and well-known public servers, with the small numbers of packets given here. Do not scan address ranges or ports of machines you do not administer.
- **What to hand in.** A short report (one file, text or PDF) with the answers to the numbered questions **Q1–Q16** and the command output that supports each answer.

The outputs printed in this sheet are illustrations with made-up addresses. Yours will differ.

Plan

Step	Activity	Time
1	Who am I? Interfaces and addresses	15 min
2	The way out: routes, neighbours, DNS servers	10 min
3	Reachability and round-trip time	15 min
4	The path: a network of networks	20 min
	Break	10 min

Step	Activity	Time
5	Names and addresses	10 min
6	A protocol by hand	20 min
7	Throughput	10 min
8	Seeing packets (optional) and wrap-up	10 min

Task 1 — Who am I?

```
hostname
ip -br link
ip -br addr
ip addr show

$ ip -br addr
lo        UNKNOWN    127.0.0.1/8 ::1/128
eth0      UP         10.20.3.57/24 fe80::5054:ff:fe12:3456/64
```

Each line is a **network interface**: the point where the host attaches to a link. `lo` is the loopback interface, a software link from the host to itself. The `-br` option gives the brief form; without it you also see the hardware (MAC) address after `link/ether` and the MTU, the largest packet the link carries.

- **Q1.** List your interfaces. Which one connects you to the laboratory network? Is it wired or wireless?
- **Q2.** Write down its IPv4 address with the prefix length (the number after the slash), its MAC address and its MTU. To which layer of the stack does each of the two addresses belong?
- **Q3.** Compare your IPv4 address with a neighbour's. Which part is the same?

Task 2 — The way out

```
ip route
ip route get 1.1.1.1
ip neigh
cat /etc/resolv.conf
resolvectl status      # on systems with systemd-resolved

$ ip route
default via 10.20.3.1 dev eth0 proto dhcp metric 100
10.20.3.0/24 dev eth0 proto kernel scope link src 10.20.3.57
```

This is your host's **forwarding table**. The second line says that addresses in your own network are reached directly on the link. The `default` line says that everything else is handed to one router, the **default gateway** — the edge router of the lecture. `ip neigh` lists the neighbours on the link whose MAC addresses the host has learned.

- **Q4.** What is the address of your default gateway? How many entries does your forwarding table have, compared with the hundreds of thousands in a core router? Why is so little enough for a host?
- **Q5.** Which DNS server does your host use? Is it in your own network?

Task 3 — Reachability and round-trip time

`ping` sends a small probe packet and waits for the echo. It reports the **round-trip time (RTT)**: there and back.

```
ping -c 10 <your default gateway>
ping -c 10 <your university's web server>
ping -c 10 www.kernel.org
ping -c 10 -s 1400 www.kernel.org

64 bytes from 192.0.2.80: icmp_seq=1 ttl=52 time=31.4 ms
...
10 packets transmitted, 10 received, 0% packet loss, time 9012ms
rtt min/avg/max/mdev = 30.9/31.6/33.8/0.8 ms
```

- **Q6.** Fill in a table with min / avg / max RTT and loss for the three destinations. Which of the four delay components explains most of the difference between them?
- **Q7.** Light in fibre covers 200 km per millisecond. From the *minimum* RTT to the far server, compute an upper bound on its distance from you. Why is it only an upper bound? Does the result fit where you think the server is? (Large sites are served from many locations; the answer may surprise you.)
- **Q8.** Did the larger packets (`-s 1400`) change the RTT? Which delay component depends on packet size?
- **Q9.** The `ttl` field in a reply starts at 64, 128 or 255 at the sender and loses one at each router. Estimate how many routers lie between you and each destination.

A host that does not answer `ping` is not necessarily down: many networks filter these probes. No reply is weak evidence.

Task 4 — The path: a network of networks

```
traceroute -n www.kernel.org
traceroute www.kernel.org
sudo traceroute -I www.kernel.org     # probes of the same kind as ping
mtr -rwc 20 www.kernel.org
```

`traceroute` sends probes that are allowed to cross only 1 router, then 2, then 3, and so on. The router at which a probe expires reports back, and so reveals its address. With `-n` you see addresses only; without it the tool also looks up the routers' names, which often contain the operator and a city or airport code. `mtr` repeats the measurement and gives loss and delay statistics for every hop.

- **Q10.** How many hops are there to the destination? Does this agree with your estimate in Q9?
- **Q11.** Using the router names and address blocks, mark where the path leaves (a) the campus network, (b) the national research network or your ISP, (c) each further network. Draw the path in the style of Figure 1 of the lecture and label each network.
- **Q12.** Between which two hops does the RTT increase most? What geographic step is that, probably?
- **Q13.** Trace the path to a server in Romania and to one on another continent. Up to which hop are the two paths identical? Why?

Lines of asterisks are routers that do not answer; some hops will show a higher delay than later hops. Both are normal — see Problem 8 of the seminar.

Task 5 — Names and addresses

```
dig www.kernel.org
dig +short www.kernel.org
dig +short AAAA www.kernel.org
dig -x <one router address from Task 4>
```

People use names; packets carry addresses. The **Domain Name System** translates between them, and it is itself a protocol that your host runs with the server found in Q5. In the full `dig` output look at the `ANSWER SECTION` and at the `Query time` line.

- **Q14.** What addresses does the name have? Is there more than one, and is there an IPv6 one (AAAA)? Run the same query twice: how does the query time change, and why?

Task 6 — A protocol by hand

First watch a program speak HTTP. Lines starting with `>` are sent by your host, lines starting with `<` are received.

```
curl -v http://example.org/ -o /dev/null
```

Now be the browser yourself. `nc` (netcat) opens a TCP connection and connects it to your keyboard and screen. Type the three lines, then press Enter on an empty line:

```
nc example.org 80
GET / HTTP/1.1
Host: example.org
Connection: close

```

If the server closes the connection before you finish typing, send the request in one piece:

```
printf 'GET / HTTP/1.1\r\nHost: example.org\r\nConnection: close\r\n\r\n' | nc
example.org 80
```

Then break the protocol on purpose: connect again and type `HELLO` followed by two Enters. Finally, in a second terminal, look at your host's open connections while the first one is still connected:

```
ss -tn
```

- **Q15.** Copy the first line of the server's reply to the correct request and to `HELLO`. Identify in your exchange the three things a protocol defines: format, order, actions. What did the server do when the format was violated?

Task 7 — Throughput

Download the test file whose address the instructor gives you and let `curl` report the average rate in bytes per second and the total time:

```
curl -o /dev/null -w 'speed: %{speed_download} B/s   time: %{time_total} s\n' <test file
URL>
```

- **Q16.** Convert the rate to Mbit/s. Repeat the measurement three times; then repeat it while several colleagues download at the same moment. Where do you think the bottleneck link is: your interface, the laboratory's uplink, or the server? What evidence do you have?

Task 8 — Seeing packets (optional)

In one terminal start a capture of five probe packets; in another run `ping -c 3` to your gateway.

```
sudo tcpdump -i any -n -c 5 icmp
sudo tcpdump -i eth0 -n -e -c 5 icmp     # -e also prints the link-layer header
```

Each line is one packet: time, source address, destination address, type. With `-e` you also see the MAC addresses — the link-layer header wrapped around the IP packet, which is encapsulation made visible. Next week's laboratory is devoted to packet capture with Wireshark.

If something does not work

Symptom	Likely cause and what to try
`command not found`	The package is not installed; see "Before you start".
`ping` works to the gateway but not beyond	A routing or filtering problem beyond your link. Check `ip route`; try another destination.
`ping` to an address works, to a name does not	Name resolution is failing. Check `/etc/resolv.conf`; try `dig @1.1.1.1 example.org`.
`traceroute` shows only asterisks	The default probes are filtered. Try `sudo traceroute -I` or `sudo traceroute -T -p 443`.
`Operation not permitted`	The command needs administrator rights; prefix it with `sudo`.

PART D · WEEK 1

Laboratory 1 (Windows): Exploring the network with PowerShell

Two-hour laboratory sheet (2 × 50 minutes) accompanying Lecture 1

This sheet covers the same ground as the Linux laboratory, on Windows 10 or 11. The concepts are identical; the tools have different names and, in places, behave differently. Knowing both is a practical skill: most servers run Linux, and most desktops you will be asked to troubleshoot run Windows.

By the end of the laboratory you should be able to

- find a Windows host's adapters, addresses, default gateway and DNS servers;
- measure round-trip time with `ping` and trace a path with `tracert` and `pathping`;
- resolve names with `Resolve-DnsName` and inspect the local DNS cache;
- test a TCP port, list open connections and find the process that owns each;
- send an HTTP request by hand from PowerShell;
- name the Linux equivalent of every command used.

Before you start

- **Environment.** Open **Windows Terminal** or **PowerShell**. Nothing needs to be installed. A few commands, marked *(admin)*, need a terminal opened with "Run as administrator".
- **Two families of tools.** The classic commands (`ipconfig`, `ping`, `tracert`, `netstat`) print text. The PowerShell cmdlets (`Get-NetIPConfiguration`, `Test-NetConnection`) return objects that can be filtered and sorted. You will use both.
- **A trap.** In the older Windows PowerShell 5.1, `curl` is an alias for a different command. Always type `curl.exe` to run the real curl that ships with Windows.
- **Getting help.** `Get-Help Test-NetConnection -Examples`, or `ping /?` for the classic commands.
- **Rules of conduct and what to hand in** are the same as for the Linux sheet: answers to **Q1–Q16** with supporting output. To save output to a file, add `| Out-File report.txt -Append` to a command.

The outputs printed in this sheet are illustrations with made-up addresses. Yours will differ.

Plan

Step	Activity	Time
1	Who am I? Adapters and addresses	15 min
2	The way out: routes, neighbours, DNS servers	10 min
3	Reachability and round-trip time	15 min
4	The path: a network of networks	20 min
	Break	10 min
5	Names and addresses	10 min

Step	Activity	Time
6	Ports, connections and a protocol by hand	20 min
7	Throughput	10 min
8	Linux and Windows side by side; wrap-up	10 min

Task 1 — Who am I?

```
hostname
ipconfig
ipconfig /all
Get-NetAdapter
Get-NetIPAddress -AddressFamily IPv4 | Format-Table InterfaceAlias, IPAddress,
PrefixLength
getmac /v

Ethernet adapter Ethernet:
   Physical Address. . . . . . . . . : 00-1A-2B-3C-4D-5E
   DHCP Enabled. . . . . . . . . . . : Yes
   IPv4 Address. . . . . . . . . . . : 10.20.3.58(Preferred)
   Subnet Mask . . . . . . . . . . . : 255.255.255.0
   Default Gateway . . . . . . . . . : 10.20.3.1
   DNS Servers . . . . . . . . . . . : 10.20.0.53
```

Windows calls interfaces **adapters**. `ipconfig /all` lists every adapter, including virtual ones created by VPN and virtualisation software; `Get-NetAdapter` shows which are Up and their link speed. The “Physical Address” is the MAC address. Windows writes the prefix as a subnet mask: 255.255.255.0 is the same as /24.

- **Q1.** List your adapters. Which one is Up and connects you to the laboratory network? What is its link speed?
- **Q2.** Write down its IPv4 address, subnet mask (and the equivalent prefix length) and MAC address. To which layer does each of the two addresses belong?
- **Q3.** Was the address typed in by hand or received automatically (“DHCP Enabled”)? From which server, and until when is it valid (“Lease Expires”)?

Task 2 — The way out

```
route print -4
Get-NetRoute -DestinationPrefix 0.0.0.0/0
Find-NetRoute -RemoteIPAddress 1.1.1.1
arp -a
Get-NetNeighbor -AddressFamily IPv4
Get-DnsClientServerAddress -AddressFamily IPv4
```

`route print` shows the host’s forwarding table. The line with network destination `0.0.0.0` and netmask `0.0.0.0` is the default route; its gateway is your edge router. `Find-NetRoute` answers the question “which route and which source address would be used to reach this destination?”. `arp -a` lists neighbours on the link and their MAC addresses.

- **Q4.** What is the address of your default gateway? Find its MAC address in the neighbour list.
- **Q5.** Which DNS servers does your host use? Are they in your own network?

Task 3 — Reachability and round-trip time

```
ping -n 10 <your default gateway>
ping -n 10 <your university's web server>
ping -n 10 www.kernel.org
ping -n 10 -l 1400 www.kernel.org
ping -4 -n 4 www.kernel.org          # force IPv4

Reply from 192.0.2.80: bytes=32 time=31ms TTL=52
...
Packets: Sent = 10, Received = 10, Lost = 0 (0% loss),
Minimum = 30ms, Maximum = 34ms, Average = 31ms
```

Windows `ping` sends four probes by default (`-n` sets the number, `-l` the size) and reports whole milliseconds, so a gateway on the same link often shows `time<1ms`.

- **Q6.** Fill in a table with minimum / average / maximum RTT and loss for the three destinations. Which delay component explains most of the difference?
- **Q7.** From the minimum RTT to the far server, compute an upper bound on its distance (200 km per millisecond in fibre, and remember the RTT is there *and back*). Does it fit where you think the server is?
- **Q8.** Did the larger packets (`-l 1400`) change the RTT? Which delay component depends on packet size?
- **Q9.** Ask a colleague for the address of their Windows machine and ping it. If there is no reply, is the machine switched off? (Windows Firewall, by default, does not answer these probes from other machines.) What does this teach you about interpreting a failed ping?

Task 4 — The path: a network of networks

```
tracert -d www.kernel.org
tracert www.kernel.org
Test-NetConnection www.kernel.org -TraceRoute
pathping -n -q 10 www.kernel.org
```

`tracert` works like Linux `traceroute`: probes that may cross only 1, 2, 3… routers, each expiring router reporting back. With `-d` names are not looked up, which is faster. `pathping` first traces the path and then measures loss at every hop; it takes a minute or two, so start it and read on.

One difference matters: `tracert` sends the same kind of probe as `ping` (ICMP echo), while Linux `traceroute` by default sends UDP packets. Networks may filter one kind and not the other, so the two tools can show different results on the same path.

- **Q10.** How many hops are there to the destination? Compare with the TTL seen in Task 3: Linux and network equipment usually start at 64 or 255, Windows at 128.
- **Q11.** Using the router names and address blocks, mark where the path leaves the campus network, the national research network or your ISP, and each further network. Draw the path in the style of Figure 1 of the lecture.
- **Q12.** Between which two hops does the RTT increase most? What geographic step is that, probably?
- **Q13.** In the `pathping` output, does any hop show loss? If an intermediate hop shows loss but the final destination does not, is there a real problem?

Task 5 — Names and addresses

```
Resolve-DnsName www.kernel.org
Resolve-DnsName www.kernel.org -Type AAAA
Resolve-DnsName <one router address from Task 4>      # reverse lookup
nslookup www.kernel.org
ipconfig /displaydns | Select-Object -First 40
Clear-DnsClientCache
```

`Resolve-DnsName` is the PowerShell counterpart of Linux `dig`; `nslookup` is the older tool found on every system. Windows keeps recent answers in a local **DNS cache**, which `ipconfig /displaydns` prints.

- **Q14.** What addresses does the name have? Is there an IPv6 one? Look at the TTL column: for how long may your host reuse the answer without asking again? Find the name in the cache, clear the cache, and check that it has gone.

Task 6 — Ports, connections and a protocol by hand

A server program is reached at an address *and* a **port** number; web servers listen on port 80 (HTTP) and 443 (HTTPS). Test whether a TCP connection can be opened:

```
Test-NetConnection example.org -Port 80
Test-NetConnection example.org -Port 81
```

Watch a program speak HTTP (> marks lines sent, < lines received):

```
curl.exe -v http://example.org/ -o NUL
```

Now be the browser yourself. Windows has no `nc`, so open the TCP connection from PowerShell and write the request into it. Paste the block as a whole:

```
$client = New-Object System.Net.Sockets.TcpClient('example.org', 80)
$stream = $client.GetStream()
$writer = New-Object System.IO.StreamWriter($stream)
$writer.NewLine = "`r`n"; $writer.AutoFlush = $true
$reader = New-Object System.IO.StreamReader($stream)
$writer.WriteLine('GET / HTTP/1.1')
$writer.WriteLine('Host: example.org')
$writer.WriteLine('Connection: close')
$writer.WriteLine('')
$reader.ReadToEnd()
$client.Close()
```

Run it again with the first request line replaced by `HELLO`. Then list the connections your machine has open, and find which program owns one of them:

```
netstat -ano | Select-String ESTABLISHED
Get-NetTCPConnection -State Established | Sort-Object RemotePort | Format-Table
LocalPort, RemoteAddress, RemotePort, OwningProcess
Get-Process -Id <a number from the OwningProcess column>
```

- **Q15.** Copy the first line of the server's reply to the correct request and to `HELLO`. Identify in your exchange the format, the order and the actions the protocol defines. What happened with port 81, and how long did the test take to give up? Why so long?

Task 7 — Throughput

```
curl.exe -o NUL -w "speed: %{speed_download} B/s   time: %{time_total} s\n" <test file
URL>
```

While the download runs, open **Task Manager → Performance** and select your adapter to watch the receive rate live.

- **Q16.** Convert the rate to Mbit/s and compare it with the link speed from Q1. Repeat three times, then while several colleagues download together. Where is the bottleneck: your adapter, the laboratory's uplink, or the server? What evidence do you have?

Task 8 — Linux and Windows side by side

Complete the right-hand column from memory before looking back; then keep the table.

Purpose	Linux	Windows (classic)	Windows (PowerShell)
Interfaces and addresses	`ip addr`	`ipconfig /all`	`Get-NetIPConfiguration`
Forwarding table	`ip route`	`route print`	`Get-NetRoute`
Route to one destination	`ip route get`	–	`Find-NetRoute`
Neighbours on the link	`ip neigh`	`arp -a`	`Get-NetNeighbor`
Reachability, RTT	`ping -c`	`ping -n`	`Test-Connection`
Path	`traceroute`, `mtr`	`tracert`, `pathping`	`Test-NetConnection -TraceRoute`
Name lookup	`dig`	`nslookup`	`Resolve-DnsName`
Test a TCP port	`nc -vz host port`	–	`Test-NetConnection -Port`
Open connections	`ss -tn`	`netstat -ano`	`Get-NetTCPConnection`
HTTP client	`curl -v`	`curl.exe -v`	`Invoke-WebRequest`
Packet capture	`tcpdump`, Wireshark	`pktmon` *(admin)*, Wireshark	–

If the Windows Subsystem for Linux is installed, every command of the Linux sheet also runs on this machine inside WSL. Try `ip addr` there and compare with `ipconfig`: the addresses differ, because WSL 2 by default sits in its own virtual network behind the Windows host.

If something does not work

Symptom	Likely cause and what to try
`Request timed out` to every destination	No connectivity, or probes filtered. Check `ipconfig` for an address and gateway; ping the gateway first.
Address begins with 169.254	The host did not receive an address automatically. Check the cable or Wi-Fi; run `ipconfig /renew`.
Ping to an address works, to a name does not	Name resolution is failing. Check Q5; try `Resolve-DnsName example.org -Server 1.1.1.1`.
`curl -v` prints an HTML object and no trace	You ran the PowerShell alias. Type `curl.exe`.
`Access is denied` or "requires elevation"	Open the terminal with "Run as administrator".

Next week

Seminar 2 treats queueing delay and the reference models with further problems. Laboratory 2, on both systems, is packet capture with Wireshark: you will see the headers of Figure 3 in real packets.